LAPORAN PRAKTIKUM

- **Identitas Praktikum**

  Nama MK : Praktikum Pemrograman Berorientasi Objek
  Kode MK : CAK2KAB4
  Bobot SKS : 3 SKS
  Tempat : L-MM
  Hari, tanggal : Jumat, 13 Maret 2026
  Jam : 12:30-15:30
  Topik praktikum : Modul-2

- **Identitas Mahasiswa**

  Nama lengkap : Harald Riandi Rantetana Purukan
  NIM : 103112400130
  Program Studi : S-1 Teknik Informatika

- **Algoritma / studi kasus (jika ada) & Penjelasan**

  1. Inisialisasi: Mesin dihidupkan dengan kode identitas, stok awal (teh, kopi, susu), dan harga masing-masing. Saldo awal dan total penjualan diatur ke nol.

  2. MasukkanUang(jumlah): - Validasi: jumlah harus lebih dari 0.

  - Aksi: Tambahkan jumlah ke saldoMasuk.

  3. PilihMinuman(jenis):

  - Evaluasi jenis (teh/kopi/susu).
  - Validasi 1 (Stok): Jika stok minuman pilihan <= 0, batalkan operasi.
  - Validasi 2 (Saldo): Jika saldoMasuk < harga minuman, batalkan operasi.
  - Eksekusi:
    - Kurangi stok minuman pilihan sebanyak 1.
    - Tambahkan harga minuman ke totalPenjualan.
    - Hitung kembalian = saldoMasuk - harga minuman.
    - Reset saldoMasuk = 0.
    - Keluarkan minuman dan kembalian.

  4. BatalkanTransaksi():

  - Jika saldoMasuk > 0, keluarkan seluruh saldoMasuk sebagai kembalian.
  - Reset saldoMasuk = 0.

  5. TampilkanStatus(): Cetak semua variabel state ke output standar.

- **Kode Program & Penjelasan**

```
class VendingMachine {
    private String kodeMesin;
    private double saldoMasuk;
    private int stokTeh;
    private int stokKopi;
    private int stokSusu;
    private double hargaTeh;
    private double hargaKopi;
    private double hargaSusu;
    private double totalPenjualan;

    public VendingMachine(String kodeMesin, int stokTeh, int stokKopi, int stokSusu,
                          double hargaTeh, double hargaKopi, double hargaSusu) {
        this.kodeMesin = kodeMesin;
        this.stokTeh = stokTeh;
        this.stokKopi = stokKopi;
        this.stokSusu = stokSusu;
        this.hargaTeh = hargaTeh;
        this.hargaKopi = hargaKopi;
        this.hargaSusu = hargaSusu;
        this.saldoMasuk = 0.0;
        this.totalPenjualan = 0.0;
    }

    public void masukkanUang(double jumlah) {
        if (jumlah <= 0) {
            System.out.println("Error: Jumlah uang tidak valid.");
            return;
        }
        this.saldoMasuk += jumlah;
        System.out.println("Uang masuk: Rp" + jumlah + " | Total Saldo: Rp" + this.saldoMasuk);
    }

    public void pilihMinuman(String jenis) {
        double hargaTerpilih = 0;
```

```
        int stokTerpilih = 0;

        jenis = jenis.toLowerCase();

        switch (jenis) {
            case "teh":
                hargaTerpilih = hargaTeh;
                stokTerpilih = stokTeh;
                break;
            case "kopi":
                hargaTerpilih = hargaKopi;
                stokTerpilih = stokKopi;
                break;
            case "susu":
                hargaTerpilih = hargaSusu;
                stokTerpilih = stokSusu;
                break;
            default:
                System.out.println("Error: Minuman tidak valid.");
                return;
        }

        if (stokTerpilih <= 0) {
            System.out.println("Transaksi Gagal: Stok " + jenis + " habis.");
            return;
        }

        if (this.saldoMasuk < hargaTerpilih) {
            System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Harga "
    + jenis + " adalah Rp" + hargaTerpilih);
            return;
        }

        // Eksekusi Transaksi
        if (jenis.equals("teh")) this.stokTeh--;
        else if (jenis.equals("kopi")) this.stokKopi--;
        else if (jenis.equals("susu")) this.stokSusu--;

        double kembalian = this.saldoMasuk - hargaTerpilih;
```

```
        this.totalPenjualan += hargaTerpilih;
        this.saldoMasuk = 0; // Reset saldo setelah transaksi

        System.out.println("Transaksi Berhasil: Mengeluarkan " + jenis + ".");
        System.out.println("Kembalian Anda: Rp" + kembalian);
    }

    public void batalkanTransaksi() {
        if (this.saldoMasuk > 0) {
            System.out.println("Transaksi dibatalkan. Mengembalikan uang: Rp"
+ this.saldoMasuk);
            this.saldoMasuk = 0;
        } else {
            System.out.println("Tidak ada saldo untuk dikembalikan.");
        }
    }

    public void tampilkanStatus() {
        System.out.println("\n=== STATUS MESIN [" + this.kodeMesin + "]
===");
        System.out.println("Stok Teh  : " + this.stokTeh);
        System.out.println("Stok Kopi : " + this.stokKopi);
        System.out.println("Stok Susu : " + this.stokSusu);
        System.out.println("Saldo Berjalan  : Rp" + this.saldoMasuk);
        System.out.println("Total Penjualan : Rp" + this.totalPenjualan);

System.out.println("================================\n");
    }
}

public class Main {
    public static void main(String[] args) {
        // Inisialisasi: Kode Mesin, Stok(Teh, Kopi, Susu), Harga(Teh, Kopi,
Susu)
        VendingMachine vm = new VendingMachine("VM-01", 1, 5, 5, 5000,
8000, 7000);

        vm.tampilkanStatus();
```

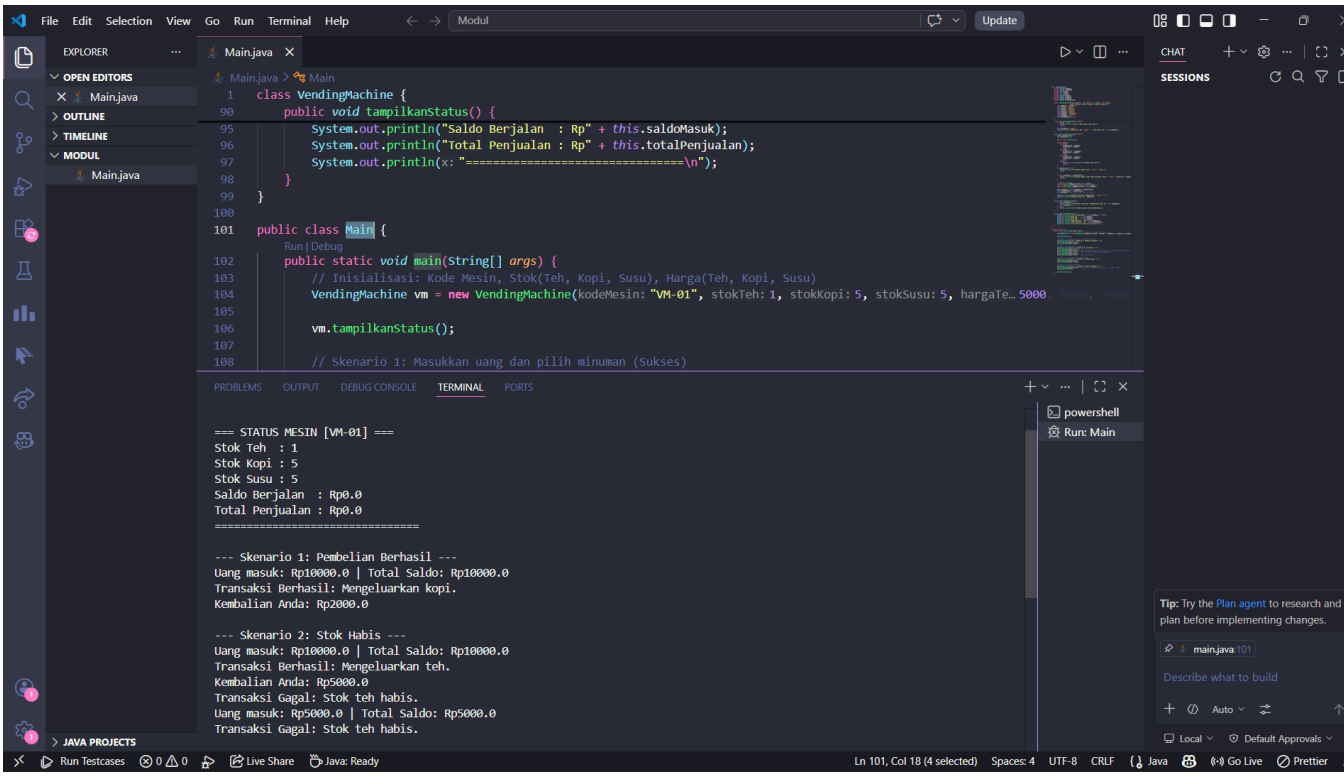
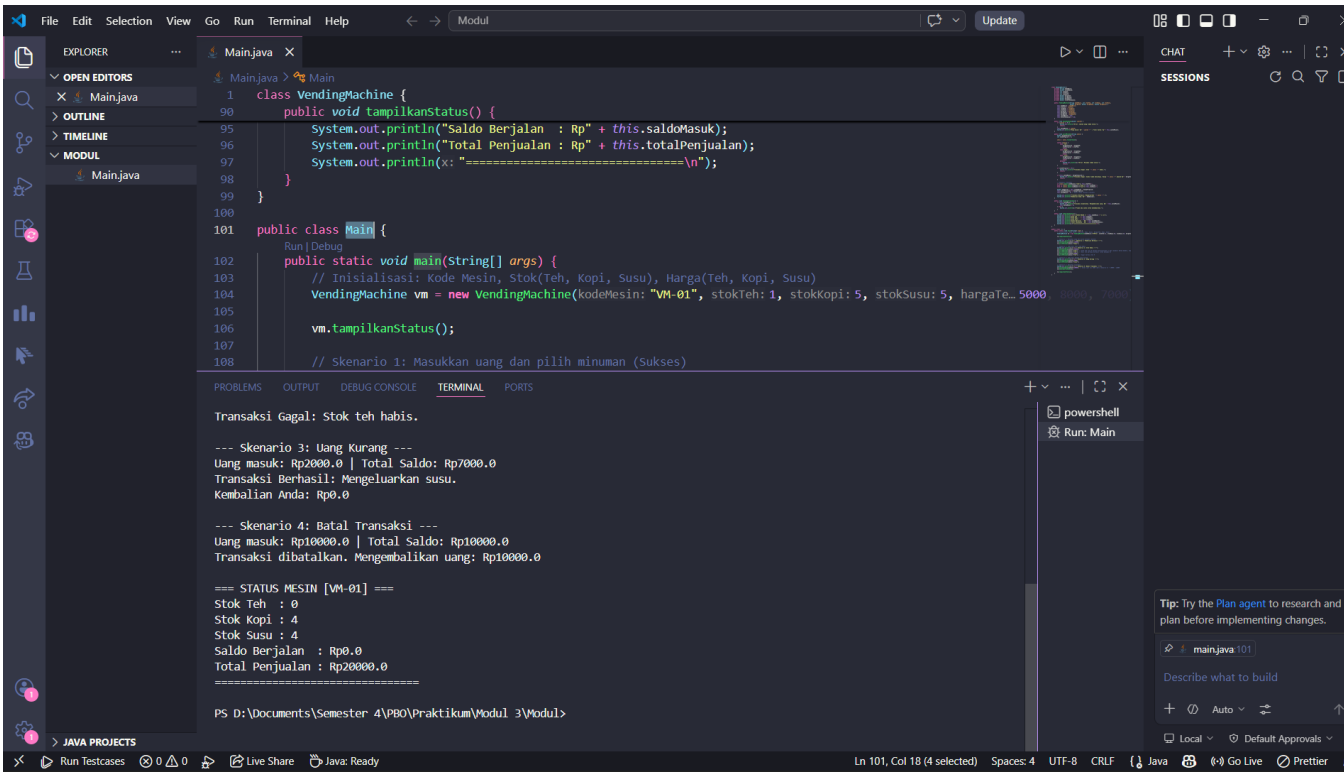
```
        // Skenario 1: Masukkan uang dan pilih minuman (Sukses)
        System.out.println("--- Skenario 1: Pembelian Berhasil ---");
        vm.masukkanUang(10000);
        vm.pilihMinuman("kopi");

        // Skenario 2: Coba beli saat stok habis
        System.out.println("\n--- Skenario 2: Stok Habis ---");
        vm.masukkanUang(10000);
        vm.pilihMinuman("teh"); // Stok teh sisa 0 karena diinisialisasi 1 dan
    (asumsi) belum dibeli, tapi mari kita beli dulu
        vm.pilihMinuman("teh"); // Beli teh pertama berhasil (stok menjadi 0)
        vm.masukkanUang(5000);
        vm.pilihMinuman("teh"); // Gagal karena stok 0

        // Skenario 3: Coba beli dengan uang kurang
        System.out.println("\n--- Skenario 3: Uang Kurang ---");
        vm.masukkanUang(2000);
        vm.pilihMinuman("susu");

        // Skenario 4: Batalkan transaksi
        System.out.println("\n--- Skenario 4: Batal Transaksi ---");
        vm.masukkanUang(10000); // Saldo saat ini 2000 (dari skenario 3) +
    10000 = 12000
        vm.batalkanTransaksi();

        vm.tampilkanStatus();
      }
}
```

Penjelasan Kode:

Enkapsulasi: Penggunaan akses modifier private pada atribut mencegah manipulasi data secara langsung dari luar kelas. Perubahan nilai atribut seperti totalPenjualan dan stok diatur ketat oleh logika dalam metode.

Manajemen State (Constructor): Constructor VendingMachine memastikan bahwa setiap objek mesin yang dibuat berada dalam kondisi valid awal yang spesifik (memerlukan parameter kode, stok, dan harga), tanpa hardcode.

Sistem Kontrol Alur (Flow Control): Pada metode pilihMinuman, penggunaan validasi awal (pengecekan stok dan saldo sebelum merubah state objek) mencegah terjadinya kondisi state tidak stabil (seperti saldo terpotong tetapi barang tidak

keluar). Evaluasi menghentikan eksekusi metode seketika menggunakan return jika kondisi syarat tidak terpenuhi.

- **Hasil Running Program**